



RescueNet

A blackboard multi-agent system for animal rescue routing

Anonymous reports from anyone, anywhere → AI agents enrich, dedupe, and match each case → a real shelter takes the animal — in minutes.

TECH STACK



React Native + Expo

Expo Router · maps · offline-first queue



AWS CDK · TypeScript

5 stacks · infrastructure as code



EventBridge + Lambda

12 event-driven serverless functions



DynamoDB + S3

blackboard case store · photo bucket



AppSync GraphQL

mutations · queries · live subscriptions



Rekognition + Bedrock

vision + LLM enrichment · Slack escalation

An AI-first blackboard architecture

Specialist agents never call each other — they watch a shared case record and react to events.

ACTORS



Reporter

anonymous, via mobile app



Coordinators

humans for edge cases



Shelters

capacity + capabilities



The Blackboard

One shared case record per animal.
Every agent reads it, writes what it knows,
and leaves a visible audit trail.

SPECIALIST AGENTS



Vision

species + injuries from photos



Geocoding

coordinates → place context



Needs profile

urgency + hard constraints



Dedup

merge duplicate sightings



Arbitrator

scores bids, assigns the case

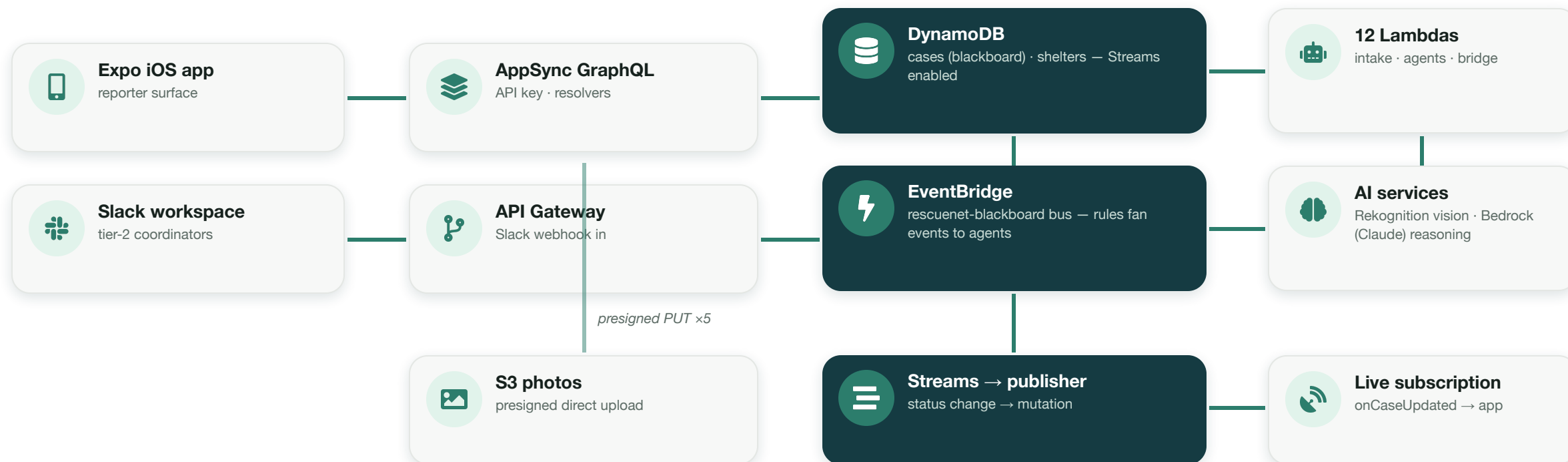


Event bus — every write announces itself; interested agents wake up

No agent-to-agent calls · add or remove an agent without touching the others · **the case record is the API**

The same shape, on AWS

Fully serverless — five CDK stacks, zero servers to keep alive between reports.



CDK STACKS

Storage — S3

Database — DynamoDB + seeds

Messaging — EventBridge

Api — AppSync + schema

Lambda — agents + resolvers

Cross-stack wiring via CDK exports — the API construct is shared so Lambda resolvers live beside their functions.

One report, end to end

From a passerby's phone to a confirmed shelter assignment — no human in the loop unless the system asks for one.



1 • Report

submitReport mutation — location, species, notes, up to 5 photos via presigned S3 URLs



2 • Case created

intake Lambda writes the case row and fires ReportSubmitted onto the bus



3 • Enrichment

vision, geocoding and needs-profile agents run in parallel, each appending to the record



4 • Dedup gate

proximity + similarity check across active cases — merge, soft-link, or clear to publish



5 • Bid window

tier-1 shelter agents score the case against live capacity and capabilities, then bid



6 • Arbitration

arbitrator picks the best bid; no eligible bids → escalate to a coordinator in Slack



7 • Confirmation

shelter coordinator confirms; declined-after-assignment cases re-route automatically



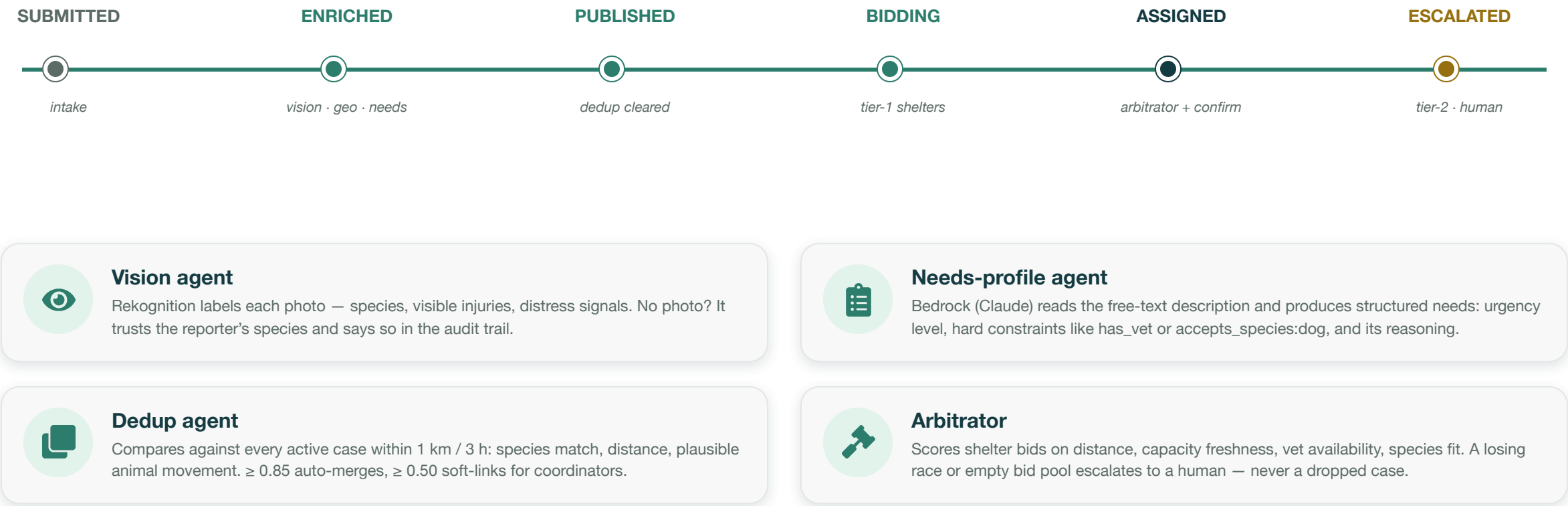
8 • Live updates

every status change streams back to the reporter's phone over a GraphQL subscription

~seconds per stage — agents react to events, nothing polls

How a case gets smarter

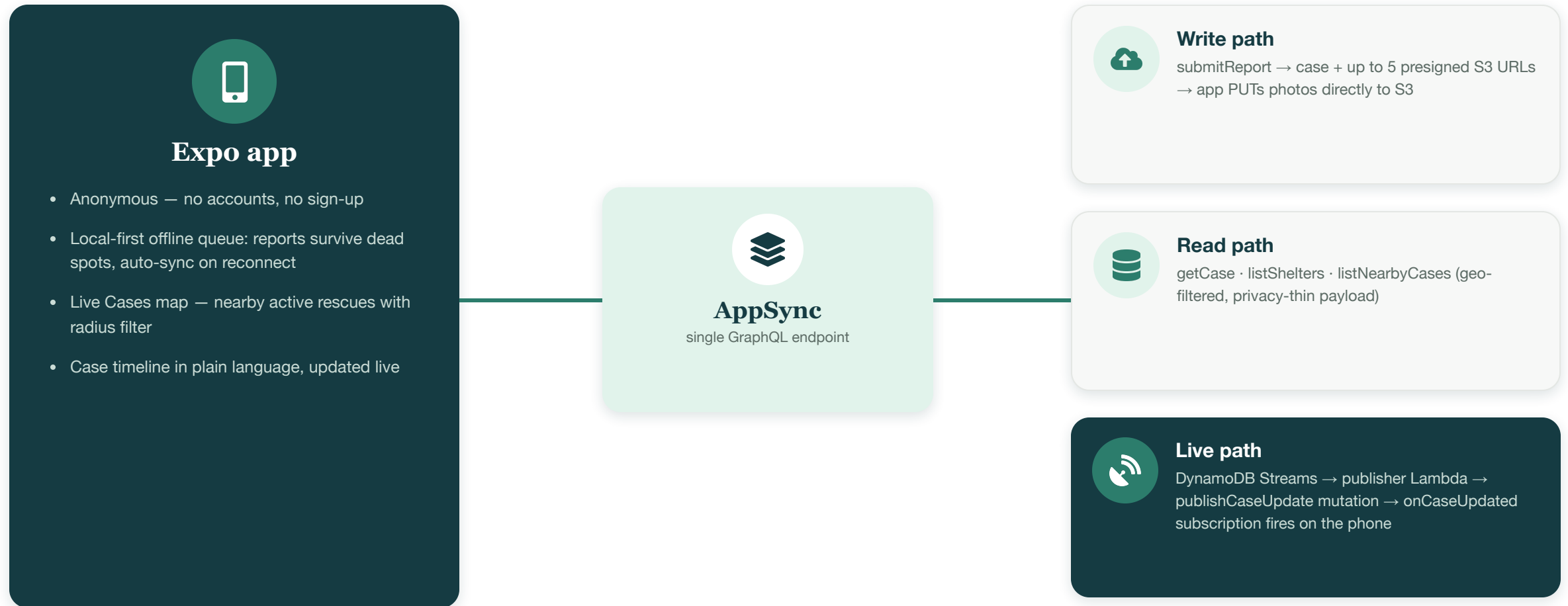
The record starts as a pin on a map and three sentences — agents layer on everything routing needs.



Every write is an eventHistory entry — the reporter sees the same trail as friendly milestones in the app; engineers see the raw agent log.

How the app talks to the blackboard

One GraphQL endpoint. Local-first writes. Push, not polling.



Why not DataStore? Creating a case is not CRUD — it mints S3 URLs and fires blackboard events. A plain GraphQL client keeps those side effects on the server, where the agents live.